

Front-End JavaScript: Make Your Web Pages Come Alive

Your working notebook for the whole course — all six modules. Read the key ideas, study the worked examples, do the exercises in the space provided, and check yourself with the quizzes.

6 modules • 22 lessons

Write-in exercises & self-checks

Name: _____

CONTENTS

MOD 1 JavaScript Foundations

- L01 Welcome to JavaScript
- L02 Linking Scripts & the Console
- L03 Variables & Data Types
- L04 Operators & Expressions

MOD 2 Control Flow & Functions

- L05 Conditionals
- L06 Loops
- L07 Functions
- L08 Scope & Hoisting

MOD 3 Working with Data

- L09 Arrays
- L10 Objects
- L11 Array Methods
- L12 Strings & Template Literals

MOD 4 The DOM

- L13 Introduction to the DOM
- L14 Selecting & Modifying Elements
- L15 Events & Event Listeners
- L16 Creating & Removing Elements

MOD 5 Building Interactive Pages

L17 Forms & Validation

L18 Timers & Animation

L19 Local Storage

L20 Working with APIs & Fetch

MOD 6 **Capstone & Next Steps**

L21 Building a Complete Project

L22 Next Steps & Best Practices

JavaScript Foundations

Your working notebook for the first four lessons. Follow along, write your code in the space provided, and check yourself with the quizzes. From Lesson 2 on you'll build one practice project, `js-practice`, and keep adding to it.

Lessons 1-4

4 lessons

Name: _____

HOW TO USE THIS WORKBOOK

Each lesson has **Key Ideas** (the reference to keep), a **Worked Example** (read it, then type it), **Your Turn** (do it — write your code in the boxes), a **Self-Check** quiz, and **Notes**. Self-check questions come from the lesson quizzes and keep the lesson's question numbers, so a set may read Q2 · Q3 · Q4. After each lesson, answer the **Journal** prompt. It takes five minutes and makes the learning stick.

OPEN THE CONSOLE — YOU'LL DO THIS EVERY LESSON

Any browser: **F12**, then the **Console** tab. Chrome/Edge: **Ctrl+Shift+J** (Windows/Linux) or **Cmd+Option+J** (Mac). Firefox: **Ctrl+Shift+K** / **Cmd+Option+K**. My browser and shortcut: _____

MODULE PROJECT TRACKER

Tick each lab as you finish it:

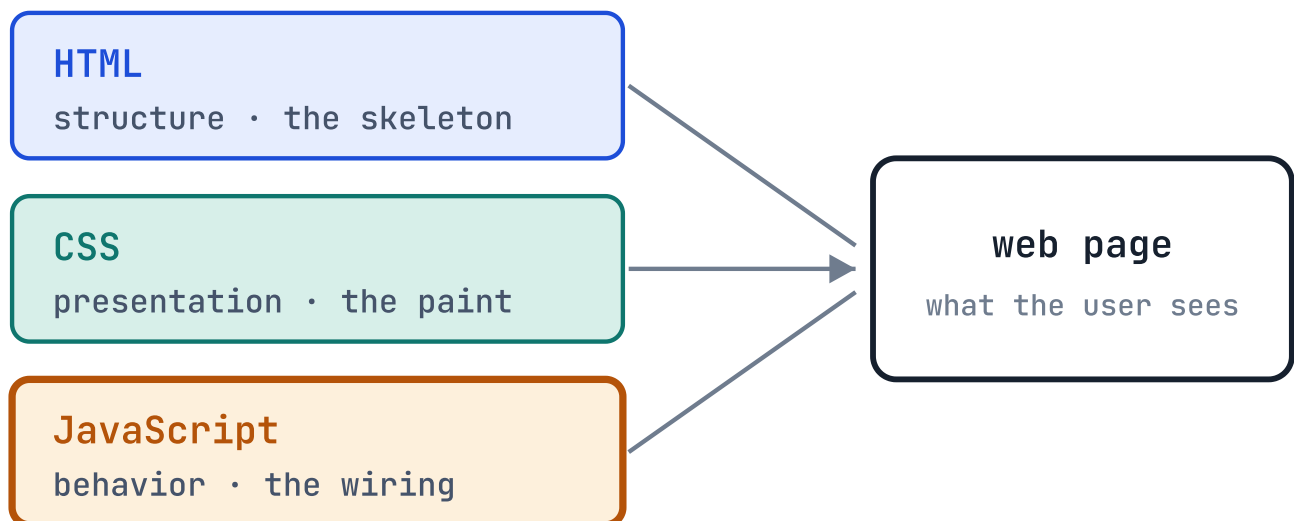
- ◆ ☐ **L01** — changed a live page's title and background from the Console
- ◆ ☐ **L02** — `js-practice` folder with `index.html` + a linked `app.js`; every console method tried
- ◆ ☐ **L03** — user profile variables logged with their types
- ◆ ☐ **L04** — tip calculator prints \$85.50 / \$17.10 / \$102.60 / \$34.20

◆ BY THE END YOU CAN

- ◆ Explain what JavaScript adds to HTML and CSS
- ◆ Tell front-end (browser) JavaScript from back-end (server) JavaScript
- ◆ Describe how a browser runs JavaScript — and why it's called **event-driven**
- ◆ Spot JavaScript at work on the sites you use

◆ KEY IDEAS

- ◆ **HTML = structure, CSS = presentation, JavaScript = behavior.**
- ◆ Every browser has a built-in **JavaScript engine** (Chrome/Edge: V8, Firefox: SpiderMonkey, Safari: JavaScriptCore). Nothing to install.
- ◆ JS runs as the page loads, then **waits for the user** and reacts to clicks, typing, and scrolling.
- ◆ **JavaScript ≠ Java**. **ECMAScript** is the official rulebook JavaScript follows (ES6 = ES2015).



Three layers, one page — you already know the first two.

◆ WORKED EXAMPLE — TYPE THESE IN THE CONSOLE

```
document.title
// → the page's title, as a string

document.title = "I changed this with JavaScript!"
// → look at the browser tab

document.body.style.backgroundColor = "lightyellow"
// → the whole page changes color. Refresh: it all comes back.
```

◆ YOUR TURN – SPOT JAVASCRIPT IN THE WILD

Run the three lines above on a site you use every day. Then use the site normally and list three features you think JavaScript is doing, and why:

◆ SELF-CHECK

Q1

What is JavaScript's primary role in web development?

- A) Structuring content on a page
- B) Styling how elements look
- C) Adding interactivity and behavior
- D) Storing data in databases

Q3

Which of these does a browser need to run JavaScript?

- A) Node.js installed on the computer
- B) A special compiler
- C) Nothing extra — browsers have a built-in JavaScript engine
- D) A browser plugin

Q4

In the exercise, what happened when you typed `document.title = "I changed this with JavaScript!"` into the Console?

- A) The text in the browser tab changed, just for your copy of the page
- B) The website's title changed for everyone who visits it
- C) The main heading (h1) on the page was replaced
- D) Nothing — the Console showed an error

◆ JOURNAL

Pick a website you use every day and list three things on it that you now think are powered by JavaScript. For each one, explain in your own words why HTML and CSS alone couldn't do it. Then name one feature you'd love to add to your own HTML & CSS site with JavaScript — and one thing about how JavaScript works that still feels fuzzy.

◆ NOTES

◆ BY THE END YOU CAN

- ◆ Link an external `.js` file with `<script src="...">`
- ◆ Print to the Console with `console.log()` and friends
- ◆ Explain where a script tag belongs and what `defer` does
- ◆ Write comments that explain *why*

◆ KEY IDEAS

- ◆ **External files win** (like a CSS stylesheet): clean HTML, reusable, cached, errors name a file and line.
- ◆ If a `<script>` has a `src`, any code *between* its tags is **ignored**.
- ◆ Put the script **just before** `</body>` (course default), or in the `<head>` with `defer`. A plain head script runs before the page exists.
- ◆ `console.log()` prints to the **Console, not the page**. Also: `.warn`, `.error`, `.info`, `.table`, `.group`, `.time`.
- ◆ Comments: `// one line` and `/* many lines */`.

TIME →

`<head>`
no defer

x JS runs before the `<h1>` exists – finds null

`before`
`</body>`

✓ every element exists (course default)

`<head>`
+ defer

`fetch` in parallel

✓ runs after parsing, in order (external files only)

Where the `<script>` tag sits decides when `app.js` runs.

◆ WORKED EXAMPLE – JS-PRACTICE

```
<!-- index.html, inside <body> -->
<h1>JavaScript Practice</h1>
<p>Open the console to see messages.</p>
<script src="app.js"></script>

// app.js
console.log("app.js is connected and running!");
console.warn("This is a warning");
console.log("5 + 3 =", 5 + 3);
console.table(["HTML", "CSS", "JavaScript"]);
```

◆ YOUR TURN – YOUR FIRST JS PROJECT

Create a `js-practice` folder with `index.html` and `app.js`, link the script, and try `log`, `warn`, `error`, `info`, two math lines, and a table. Keep this folder — you'll use it in L03 and L04. Write down what you saw in the Console:

◆ SELF-CHECK

Q1

Where is the recommended place to put a `<script>` tag that links to an external file?

- A) Inside the `<head>` (without `defer`)
- B) Just before the closing `</body>` tag
- C) After the `</html>` tag
- D) Inside the `<title>` tag

Q3

What does the `defer` attribute do on a `<script>` tag?

- A) Prevents the script from running
- B) Makes the script run faster
- C) Downloads the script in parallel and runs it after HTML parsing completes
- D) Converts the script to inline

Q4

What happens with this code?

```
<script src="app.js">
  console.log("Hi!");
</script>
```

- A) Both `app.js` and the `console.log()` run
- B) Only the `console.log()` runs
- C) The browser shows a syntax error
- D) `app.js` runs, but the `console.log()` inside the tags is ignored

◆ JOURNAL

Imagine a script in the `<head>` (without `defer`) tries to find your page's `<h1>`. Explain in your own words what goes wrong, and describe two different ways to fix it. Which approach will you use on your own site, and why? What's still unclear about when scripts run?

◆ NOTES

◆ BY THE END YOU CAN

- ◆ Declare variables with `const` and `let` (and know why to avoid `var`)
- ◆ Name the everyday types: string, number, boolean, null, undefined
- ◆ Check a type with `typeof`
- ◆ Build strings with template literals: ``Hi ${name}``

◆ KEY IDEAS – LABELED BOXES

- ◆ `const` by default, `let` when the value will change, `var` only in old code. Reassigning a `const` throws a `TypeError`.
- ◆ `=` means **store** ("gets"), not "equals."
- ◆ Names: letters, digits, `_`, `$`; can't start with a digit; case-sensitive; no reserved words. Use **camelCase**.
- ◆ `null` = deliberately empty; `undefined` = not assigned yet. `typeof null` is `"object"` – a famous bug.
- ◆ `"5" + 3` is `"53"` (joins), but `"5" - 3` is `2`. Convert on purpose with `Number()` / `String()`.

`const firstName``"Ray"``x can't reassign``let age``30``31``let nickname``null``"Ace"`

A variable is a labeled box – `const` locks the label to its value; `let` can be refilled.

◆ WORKED EXAMPLE

```
const firstName = "Ray";
let age = 30;
let nickname = null;

console.log(`firstName: ${firstName} (${typeof firstName})`); // firstName: Ray (st
console.log(`Hello, ${firstName}! You are ${age} years old.`);

age = 31; // ✓ let can change
// firstName = "Raymond"; ✗ TypeError: Assignment to constant variable
```

◆ YOUR TURN – BUILD A USER PROFILE

In `js-practice/app.js`, create `firstName`, `lastName`, `age`, `email`, `isPremiumMember`, `accountBalance`, and `nickname`. Log each with its type, print a greeting, change the `let` values, and try changing a `const`. Plan your variables here — type, and `const` or `let`:

◆ SELF-CHECK

Q2

What is the result of `typeof null`?

A) "null"

B) "object"

C) "undefined"

D) "boolean"

Q3

What does `"5" + 3` evaluate to?

A) 8

B) "53"

C) Error

D) NaN

Q4

You write `let favoriteColor;` and then `console.log(favoriteColor);`. What appears in the console?

A) undefined

B) null

C) ReferenceError: favoriteColor is not defined

D) "" (an empty string)

◆ JOURNAL

Think about your own website from the HTML & CSS course. List five pieces of information it could store in variables (for example, your name, a visit counter, whether dark mode is on). For each one, choose its data type and decide between `const` and `let` — and explain why. Then describe the difference between `null` and `undefined` in your own words. Which part still feels shaky?

◆ NOTES

◆ BY THE END YOU CAN

- ◆ Do math with `+`, `-`, `*`, `/`, `%`, `**` and update values with `+=`, `*=` ...
- ◆ Compare with `===` and `!==` — and explain why not `==`
- ◆ Combine conditions with `&&`, `||`, `!`
- ◆ Predict an expression's result using precedence (or parentheses)

◆ KEY IDEAS

- ◆ `%` is the **remainder**, not percent: `10 % 3` is `1`.
- ◆ `===` checks value **and** type: `5 === "5"` is `false`. `==` converts types first and surprises you. Always use `===`.
- ◆ Falsy: `false`, `0`, `""`, `null`, `undefined`, `NaN`. Everything else is truthy — even `"0"`.
- ◆ `&&` runs before `||`; `*` before `+`. Parentheses make it obvious.
- ◆ Money: `.toFixed(2)` turns `34.199999999999996` into `"34.20"`.

◆ WORKED EXAMPLE

```
const price = 29.99;
const quantity = 3;
const subtotal = price * quantity;
console.log(`Subtotal: ${subtotal.toFixed(2)}`); // Subtotal: $89.97

let age = 25;
let hasID = true;
console.log(age >= 21 && hasID); // true
console.log(5 === "5"); // false
```

◆ YOUR TURN — TIP CALCULATOR

In `js-practice/app.js`: `billAmount = 85.50`, `tipPercentage = 20`, `numberOfPeople = 3`.

Calculate the tip, the total, and the amount per person, and log them with template literals. Your output should read **Bill: \$85.50 · Tip (20%): \$17.10 · Total: \$102.60 · Per person: \$34.20**. Bonus: log whether the total is over \$100. Write your calculations here:

◆ SELF-CHECK

Q2

What does `5 === "5"` return?

- A) false
- B) true
- C) Error
- D) undefined

Q3

What is the result of `true && false || true` ?

- A) false
- B) true
- C) undefined
- D) Error

Q4

After this code runs, what is the value of `points` ?

```
let points = 10;
points += 5;
points *= 2;
```

- A) 15
- B) 20
- C) 30
- D) 25

◆ JOURNAL

Before running them, predict what `"5" + 3`, `"5" - 3`, `5 == "5"`, and `5 === "5"` will give you, and write down why. Then check your predictions in the console. Which one surprised you? In your own words, explain why this course tells you to always use `===`. Finally, think of a rule on your own site you could write with `&&` or `||` (for example, "show the banner if it's the weekend OR a holiday") — what's still confusing about combining conditions?

◆ NOTES

YOU FINISHED MODULE 1 🎉

You can now run JavaScript in the Console and from a linked file, store data in variables, check its type, and calculate and compare with operators. In Module 2 your code starts making decisions, repeating work, and organizing itself into functions.

Control Flow & Functions

Your working notebook for the four lessons that teach your programs to make decisions, repeat actions, and organize code into reusable functions. Follow along, do the exercises in the space provided, and check yourself with the quizzes.

Lessons 05-08

4 Lessons

Name: _____

HOW TO USE THIS WORKBOOK

Each lesson has **Key Ideas** (the reference to keep), a **Worked Example** (read it), **Your Turn** (do it — write your code in the boxes), a **Self-Check** quiz, and **Notes**. The lined boxes are for you to write real code and answers. Self-check questions come from the lesson quizzes and keep the lesson's question numbers, so a set may read Q1 · Q4 · Q5. After each lesson, answer the **Journal** prompt. It takes five minutes and makes the learning stick.

MODULE LAB TRACKER

This module has **four short console programs**, one per lesson. Each lab page shows your output on the page and in the Console. Tick each lab as you finish it:

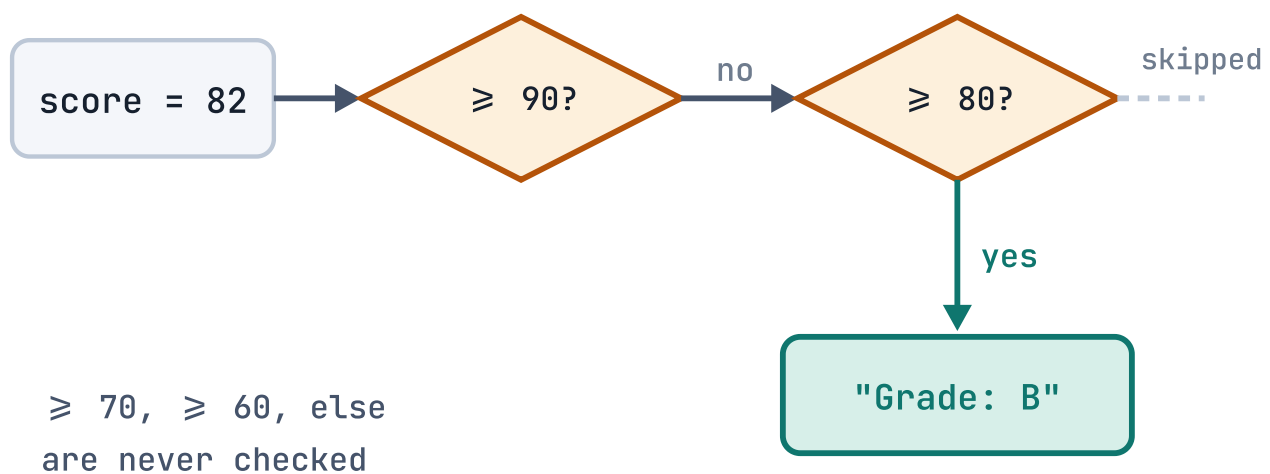
- ◆ ☐ **L05** — Grade Calculator: letter grade, message, pass/fail, invalid-score check
- ◆ ☐ **L06** — Star Rating (★★★★☆☆) + a case-insensitive search filter with a match count
- ◆ ☐ **L07** — Mini Utility Library: capitalize, clamp, pluralize, temperatureConverter
- ◆ ☐ **L08** — Scope Detective (8 predictions) + a closure-based score tracker

◆ BY THE END YOU CAN

- ◆ Write `if`, `else if`, and `else` to control program flow
- ◆ Use `switch` to compare one value against many options
- ◆ Pick between two values with the ternary operator (`? :`)
- ◆ Use guard clauses and truthy/falsy checks for cleaner conditions

◆ KEY IDEAS

- ◆ An `else if` chain is checked **top to bottom**; the **first true condition wins** and the rest are skipped. Check the highest range first.
- ◆ `switch` compares with **strict equality** (`===`), so `"5"` never matches `case 5`. End each case with `break`, or it falls through.
- ◆ Ternary: `condition ? valueIfTrue : valueIfFalse`. Keep it short; never nest them.
- ◆ **Guard clauses** handle the problem cases first, so the main logic stays flat.
- ◆ `0` and `""` are falsy. If 0 is a valid value, don't test it with `if (quantity)`.



An `else if` chain stops at the first true condition. That's why the highest range goes first.

◆ WORKED EXAMPLE

```
const score = 85;
let letter;

if (score >= 90) {
  letter = "A";
} else if (score >= 80) {
  letter = "B";
} else {
  letter = "C or below";
}

const result = score >= 60 ? "Pass" : "Fail";
console.log(`${score} → ${letter} (${result})`); // "85 → B (Pass)"
```

◆ YOUR TURN — GRADE CALCULATOR

Turn a score into a letter grade with `if` / `else if`, pick a message with a `switch` on the letter, decide pass/fail (60+) with a ternary, and print *"Alex scored 78 (C) — Solid effort, keep improving!"* and *"Status: Pass"*. Bonus: reject scores below 0 or above 100 first. Write your grade chain and your `switch` here:

◆ SELF-CHECK

Q1

What does this code output?

```
let x = 5;
if (x > 10) {
  console.log("A");
} else if (x > 3) {
  console.log("B");
} else if (x > 1) {
  console.log("C");
} else {
  console.log("D");
}
```

A) A

B) B

C) C

D) D

Q4

What does this code output?

```
let input = "5";
switch (input) {
  case 5:
    console.log("number five");
    break;
  default:
    console.log("no match");
}
```

A) number five

B) An error

C) no match

D) Nothing prints

Q5

A shopping cart stores `let quantity = 0;` and zero is a perfectly valid quantity. Why is `if (quantity) { ... }` a risky way to check that a quantity was entered?

- A) You can't put a number inside an if condition
- B) 0 is truthy, so the check always passes
- C) It only works if quantity was declared with const
- D) 0 is falsy, so a valid 0 would be treated as "missing"

◆ JOURNAL

Think of one decision a page you'd like to build has to make (a greeting that changes with the time of day, a form that checks for a blank field, a price that changes for members). Write it out in plain English as an `if` / `else if` / `else` chain. Would a `switch` or a ternary fit it better, and why? Which of the three are you still unsure when to reach for?

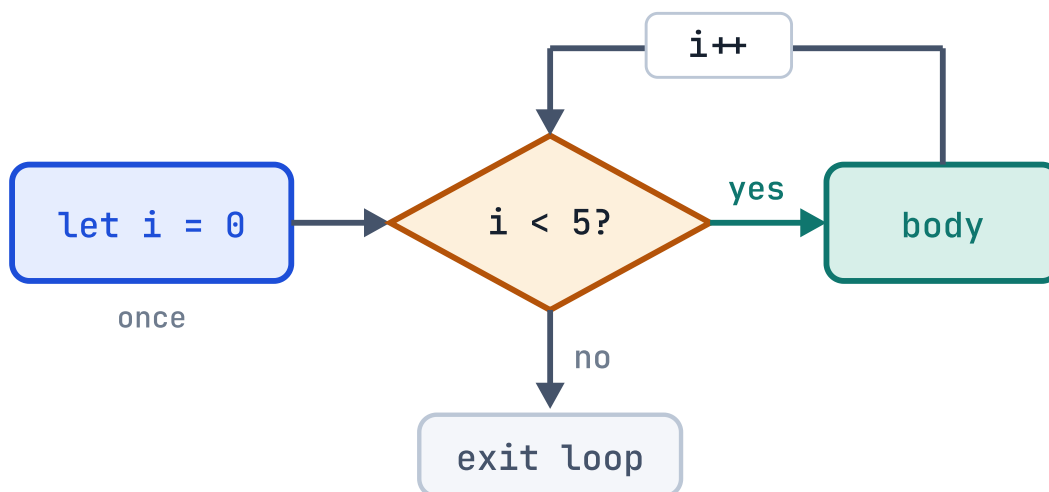
◆ NOTES

◆ BY THE END YOU CAN

- ◆ Write a `for` loop that repeats a set number of times
- ◆ Use `while` and `do...while` when the count isn't known
- ◆ Loop array values with `for...of` and object keys with `for...in`
- ◆ Control a loop with `break` and `continue`; avoid infinite loops and off-by-one errors

◆ KEY IDEAS

- ◆ **DRY — Don't Repeat Yourself.** Write the instructions once; the loop repeats them.
- ◆ `for` when you know how many times; `while` when a condition decides; `do...while` runs at least once.
- ◆ `for...of` → array values. `for...in` → object keys (not for arrays).
- ◆ `break` exits the loop; `continue` skips to the next pass.
- ◆ Index loops use `i < array.length` — `<=` runs one extra time and reads `undefined`.



`for (let i = 0; i < 5; i++):` start once, check before every pass, update after every pass.

◆ WORKED EXAMPLE

```
const numbers = [12, 35, 8, 67, 42];

for (const num of numbers) {
  if (num > 50) {
    console.log(`Found it: ${num}`); // "Found it: 67"
    break;                          // stop searching
  }
}
```

◆ YOUR TURN – STAR RATING & SEARCH FILTER

Part 1: finish `starRating(rating)` so `starRating(3)` returns `"★★★☆☆"`. Part 2: show every product that contains `"wireless"` (any capitalization), then *"3 results found."* Write your two loops here:

◆ SELF-CHECK

Q1

How many times does this loop run?

```
for (let i = 0; i < 3; i++) {
  console.log(i);
}
```

A) 2

B) 3

C) 4

D) 1

Q3

What does `continue` do inside a loop?

- A) Exits the loop entirely
- B) Skips to the next iteration
- C) Skips the code after the loop
- D) Restarts the loop from the beginning

Q5

What goes wrong with this loop?

```
const items = ["a", "b", "c"];
for (let i = 0; i <= items.length; i++) {
  console.log(items[i]);
}
```

- A) It's an infinite loop
- B) It runs one extra time and logs undefined
- C) It throws an error on the last pass
- D) It skips the first item

◆ JOURNAL

In your own words, explain when you'd pick `for`, `while`, `for...of`, and `for...in`. Then name one list on a page you'd like to build (products, photos, to-dos, quiz questions) and describe which loop you'd use to go through it. What still trips you up: off-by-one errors, `break` vs. `continue`, or something else?

◆ NOTES

◆ BY THE END YOU CAN

- ◆ Write a function declaration and call it (even before its line — hoisting)
- ◆ Write function expressions and arrow functions
- ◆ Use parameters, arguments, and default values
- ◆ Return a value and use it in an expression

◆ KEY IDEAS

- ◆ A function is a **recipe**: define it once, call it with `()` whenever you need it. `greet` is the function; `greet()` runs it.
- ◆ **Parameters** are the parking spots in the definition; **arguments** are the cars you pass in. A missing argument is `undefined` unless it has a default.
- ◆ `return` hands a value back and exits immediately. No `return` → `undefined`.
- ◆ Declarations are hoisted; `const` expressions and arrows can't be called before their line.
- ◆ One-expression arrows return automatically: `(a, b) => a + b`. To return an object, wrap it: `n => ({ name: n })`.

◆ WORKED EXAMPLE

```
// Declaration - hoisted
function add(a, b) {
  return a + b;
}

// Expression - define before calling
const multiply = function(a, b) {
  return a * b;
};

// Arrow with a default parameter - implicit return
const greet = (name = "stranger") => `Hello, ${name}!`;

console.log(add(10, 20) * 2); // 60
console.log(multiply(4, 5)); // 20
console.log(greet()); // "Hello, stranger!"
```

◆ YOUR TURN — MINI UTILITY LIBRARY

Write `capitalize(str)` as a declaration, `clamp(value, min, max)` as an expression, `pluralize(count, singular, plural)` as an arrow with a default `plural`, and `temperatureConverter(value, fromUnit)` in any style (`"32°F = 0.0°C"`). Write each function's first line and its `return` here:

◆ SELF-CHECK

Q1

What does a function return if it has no `return` statement?

A) null

B) undefined

C) 0

D) Nothing (error)

Q2

Which can be called before it's defined in the code?

A) Function declaration

B) Function expression

C) Arrow function

D) All of the above

Q5

Which arrow function correctly returns an object?

A) `const make = n => { name: n };`

B) `const make = n => return { name: n };`

C) `const make = n => [name: n];`

D) `const make = n => ({ name: n });`

◆ JOURNAL

Pick one small job you found yourself repeating in Lessons 5 and 6 and describe a function that would handle it: its name, its parameters, and what it returns. Would you write it as a declaration or an arrow function, and why? What is still fuzzy for you: parameters vs. arguments, `return`, or the arrow shortcuts?

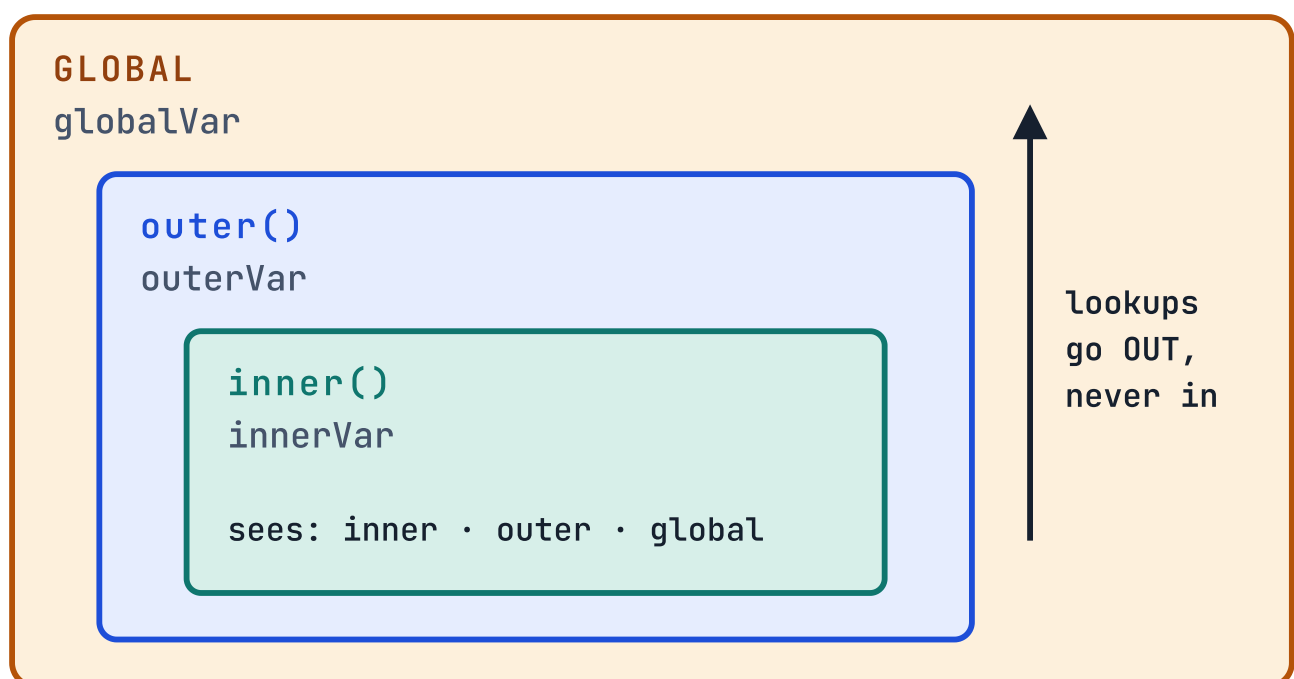
◆ NOTES

◆ BY THE END YOU CAN

- ◆ Tell global, function, and block scope apart
- ◆ Predict where a variable can and can't be used
- ◆ Explain hoisting for `var`, `let` / `const`, and functions, including the Temporal Dead Zone
- ◆ Explain what a closure is and build one

◆ KEY IDEAS

- ◆ `let` / `const` are **block-scoped** (they stay inside `{ }`). `var` is **function-scoped** and leaks out of `if` blocks and loops.
- ◆ The **scope chain**: JavaScript looks in the current scope, then outward, up to global. It never looks *inward*.
- ◆ **Shadowing**: an inner variable with the same name hides the outer one inside its scope only.
- ◆ **Hoisting**: `var` starts as `undefined`; `let` / `const` throw a `ReferenceError` before their line (the TDZ); function declarations are fully hoisted.
- ◆ A **closure** is a function that remembers the variables of the scope it was made in, even after that outer function has finished.



The scope chain: `inner()` can use `outerVar` and `globalVar`, but `outer()` can't see `innerVar`.

◆ WORKED EXAMPLE

```
function createCounter() {
  let count = 0; // private – only the methods below can reach it

  return {
    increment() { count++; },
    getCount() { return count; }
  };
}

const counter = createCounter();
counter.increment();
counter.increment();
console.log(counter.getCount()); // 2
console.log(counter.count);     // undefined – count is not a property
```

◆ YOUR TURN – SCOPE DETECTIVE & SCORE TRACKER

Part 1: before you run the lab, write your prediction for each of the eight outputs:

LINE	MY PREDICTION	ACTUAL
1		
2		
3		
4		
5		
6		
7		
8		

Part 2: write `createScoreTracker()` with `addPoints`, `penalty` (never below 0), `getScore`, and `reset`:

Q1

What is the output?

```
console.log(x);  
var x = 10;
```

- A) 10
- B) undefined
- C) ReferenceError
- D) null

Q3

What is a closure?

- A) A function inside curly braces
- B) A variable that can be accessed globally
- C) A function that remembers variables from its outer scope
- D) A way to call a function immediately

Q4

What happens when this code runs?

```
console.log(total);  
let total = 50;
```

- A) It logs undefined
- B) ReferenceError: Cannot access 'total' before initialization
- C) It logs 50
- D) It logs null

Without looking back, explain in your own words why `var` leaks out of an `if` block but `let` doesn't, and what the Temporal Dead Zone protects you from. Then describe a closure as if you were explaining the counter example to a friend. Which part of this lesson do you expect to re-read?

◆ NOTES

YOU FINISHED MODULE 2 🎉

Your programs can now make decisions, repeat actions, package logic into reusable functions, and keep variables where they belong. In Module 3 you'll give them real data to work with — arrays, objects, and strings.

Working with Data

Your working notebook for the four data lessons: arrays, objects, array methods, and strings. Follow along, do the exercises in the space provided, and check yourself with the quizzes.

Lessons 9-12

4 Lessons

Name: _____

HOW TO USE THIS WORKBOOK

Each lesson has **Key Ideas** (the reference to keep), a **Worked Example** (read it), **Your Turn** (do it — write your code in the boxes), a **Self-Check** quiz, and **Notes**. The lined boxes are for you to write real code and answers. Self-check questions come from the lesson quizzes and keep the lesson's question numbers, so a set may read Q1 · Q3 · Q4. After each lesson, answer the **Journal** prompt. It takes five minutes and makes the learning stick.

MODULE LAB TRACKER

Each lesson has its own lab page. Your code runs in `script.js`, and a helper called `show()` prints every result on the page *and* in the Console. Use `show()` wherever the lesson says `console.log()`. Tick each lab as you finish it:

- ◆ ☐ **L09** — To-Do List Manager: add, remove, insert, list, complete
- ◆ ☐ **L10** — Contact Book: add, find, update, display, list by group
- ◆ ☐ **L11** — Product Catalog Pipeline: six challenges, no loops
- ◆ ☐ **L12** — Text Formatting Utility: truncate, slugify, countWords, initials, maskEmail

◆ BY THE END YOU CAN

- ◆ Create an array and read or change any element by **index** (starting at 0)
- ◆ Add and remove with `push`, `pop`, `unshift`, `shift`
- ◆ Insert, remove, or replace anywhere with `splice`
- ◆ Search with `includes`, `indexOf`, `find`; copy with `slice` and spread
- ◆ Loop with `for`, `for...of`, and `forEach`

◆ KEY IDEAS

- ◆ The first element is `arr[0]`; the last is `arr[arr.length - 1]` or `arr.at(-1)`. Out of range gives `undefined`, not an error.
- ◆ `push/pop` work on the end; `unshift/shift` on the start. `push` returns the new length; `pop` and `shift` return the removed item.
- ◆ `indexOf` returns -1 when the value isn't there. Check for it before you splice.
- ◆ `splice` changes the original; `slice` makes a copy. Same-sounding names, opposite effects.
- ◆ A `const` array can still change its contents. `const` only stops you reassigning the variable.

letters.slice(1, 3)

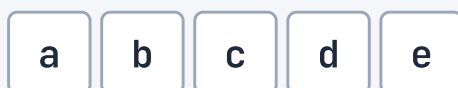
letters (before)



returns a NEW array



letters (after): unchanged



letters.splice(1, 2)

letters (before)



returns the removed items



letters (after): CHANGED



slice copies a portion out; splice cuts it out of the original.

◆ WORKED EXAMPLE

```
const colors = ["red", "yellow", "purple"];
colors.splice(1, 0, "orange"); // insert: ["red", "orange", "yellow", "purple"]
colors.push("blue");           // add to end
const i = colors.indexOf("yellow"); // 2
if (i !== -1) colors.splice(i, 1); // remove it safely

for (const color of colors) {
  console.log(color);           // red, orange, purple, blue
}
```

◆ YOUR TURN – TO-DO LIST MANAGER

Open `109-arrays/starter/`. Write `addTodo`, `removeTodo`, `insertTodoAt`, `listTodos`, and `completeTodo`, then uncomment the tests. Plan it first: which array method does each function need?

◆ SELF-CHECK

Q1

What is the index of the first element in a JavaScript array?

A) 0

B) 1

C) -1

D) It depends on the array

Q3

What does `splice()` do?

- A) Creates a copy of part of an array
- B) Joins two arrays together
- C) Removes, inserts, or replaces elements at any position
- D) Sorts the array in place

Q4

What does `["a", "b", "c"].includes("b")` return?

- A) 1
- B) true
- C) false
- D) "b"

◆ JOURNAL

Think of a list on a page you'd like to build — a playlist, a reading list, a set of scores. Which array methods would you use to add, remove, and find items in it? In your own words, explain the difference between `splice` and `slice`, and note which method you're still unsure about.

◆ NOTES

◆ BY THE END YOU CAN

- ◆ Create objects and read, add, update, and delete properties
- ◆ Choose dot notation (default) or bracket notation (odd keys, keys in variables)
- ◆ Write methods that use `this`
- ◆ Work with nested data and arrays of objects; destructure properties
- ◆ Loop over `Object.keys`, `values`, and `entries`

◆ KEY IDEAS

- ◆ An object stores **key: value** pairs, so you look data up by *name* instead of position.
- ◆ `obj[field]` uses the *value* of `field`; `obj.field` looks for a key literally named "field".
- ◆ Inside a method, `this` is the object before the dot. **Arrow functions don't have their own `this`**, so write methods as `greet() { ... }`.
- ◆ An **array of objects** is like a spreadsheet: each object is a row, each key a column.
- ◆ Objects are compared by **reference**: two objects with the same contents are still not `===`.

variables

a

c

b

object #1 (in memory)

{ x: 1, y: 2 }

object #2 (in memory)

{ x: 1, y: 2 }

a === c → true

a === b → false

=== on objects asks "same object?", not "same contents?"

◆ WORKED EXAMPLE

```
const player = {
  name: "Ray",
  score: 0,
  addPoints(points) {           // method: regular syntax, not an arrow
    this.score += points;
    return `${this.name}: ${this.score} points`;
  }
};
player.addPoints(10);           // "Ray: 10 points"

const { name, score } = player; // destructuring
for (const [key, value] of Object.entries({ name, score })) {
  console.log(`${key} → ${value}`); // name → Ray, score → 10
}
```

◆ YOUR TURN – CONTACT BOOK

Open `l10-objects/starter/`. Write `addContact`, `findContact`, `updateContact`, `displayContact`, and `listByGroup`. A `for...of` loop is fine for `listByGroup`; you'll meet the shortcut next lesson. Sketch `findContact` and `updateContact` here:

◆ SELF-CHECK

Q1

Which notation must you use when the key is stored in a variable?

A) Dot notation

B) Bracket notation

C) Parentheses notation

D) Either works

Q3

Why shouldn't you use arrow functions as object methods that use `this` ?

- A) Arrow functions can't be properties
- B) Arrow functions don't have their own `this`
- C) Arrow functions can't return values
- D) Arrow functions cause syntax errors in objects

Q4

What does `{ x: 1 } === { x: 1 }` evaluate to?

- A) true
- B) false
- C) TypeError
- D) It depends on the values

◆ JOURNAL

Describe something from your own site or life as an object — a recipe, a pet, a product — with at least four properties and one method. When would you reach for bracket notation instead of dot notation? What part of `this` or destructuring still feels fuzzy?

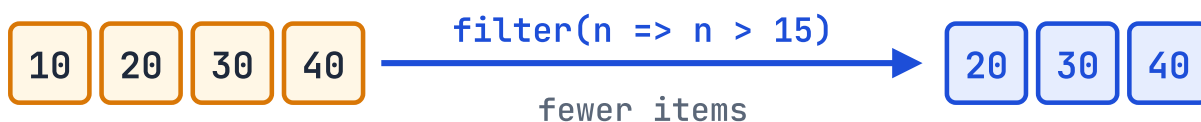
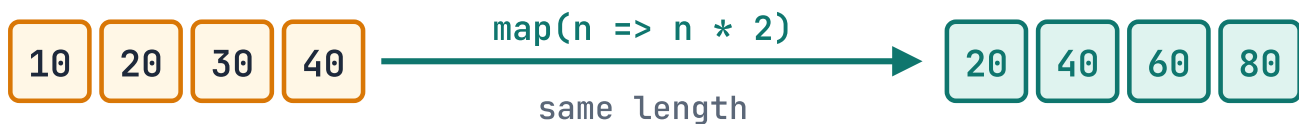
◆ NOTES

◆ BY THE END YOU CAN

- ◆ Transform with `map`, keep matches with `filter`, collapse with `reduce`
- ◆ Ask yes/no questions with `some` and `every`; get the first match with `find`
- ◆ Sort numbers and objects with a compare function
- ◆ Chain methods into a filter → sort → map pipeline

◆ KEY IDEAS

- ◆ `map` → new array, same length · `filter` → new array, fewer items · `reduce` → one value. `forEach` returns `undefined`; use it for side effects only.
- ◆ Always give `reduce` an initial value: `reduce((sum, n) => sum + n, 0)`.
- ◆ Plain `sort()` sorts as text: `[40, 100, 1, 5, 25] → [1, 100, 25, 40, 5]`. For numbers use `(a, b) => a - b`.
- ◆ `sort` and `reverse` change the original; `toSorted` or `[...arr].sort()` don't.
- ◆ Money math gives long decimals like `374.94000000000005`. Display with `toFixed(2)`, which returns a string.



map keeps the length · filter keeps the matches · reduce collapses to one value

◆ WORKED EXAMPLE

```
const result = products
  .filter(p => p.category === "electronics") // keep electronics
  .sort((a, b) => a.price - b.price)         // cheapest first
  .map(p => `${p.name}: ${p.price}`);       // format
// ["Tablet: $449", "Phone: $699", "Laptop: $999"]

const total = products
  .filter(p => p.category === "electronics")
  .reduce((sum, p) => sum + p.price, 0);    // 2147
```

◆ YOUR TURN – PRODUCT CATALOG PIPELINE

Open `111-array-methods/starter/`. Solve the six challenges with array methods only (no `for` / `while`). Before coding each one, write the chain in plain words, e.g. "filter in-stock → reduce to a sum":

◆ SELF-CHECK

Q1

What does `map()` return?

- A) A single value
- B) A new array with the same number of elements
- C) A new array with fewer elements
- D) undefined

Q2

What does `[1,2,3].filter(n => n > 1)` return?

A) true

B) [1, 2, 3]

C) [2, 3]

D) [2]

Q4

Which method modifies the original array?

A) map()

B) filter()

C) sort()

D) reduce()

◆ JOURNAL

Pick a small dataset you care about (your movies, workouts, or expenses) and write, in plain English, a filter → sort → map pipeline you'd run on it. Why does `filter` usually come before `map`? Which method — probably `reduce` — are you still unsure about?

◆ NOTES

◆ BY THE END YOU CAN

- ◆ Build text with template literals: ``Hi, ${name}!`` and multiline strings
- ◆ Search with `includes`, `startsWith`, `endsWith`, `indexOf`
- ◆ Extract with `slice` and `at`; clean with `trim`, `toLowerCase`, `replace`
- ◆ Split text into arrays and join arrays back into text
- ◆ Format with `padStart`, `padEnd`, and `repeat`

◆ KEY IDEAS

- ◆ **Strings are immutable.** Every method returns a *new* string, so store the result: `name = name.trim();`
- ◆ Only **backticks** interpolate `${...}`. Regular quotes print it literally.
- ◆ `slice` works like array slice, and negatives count from the end: `"Hello, World!".slice(-6) → "World!"`.
- ◆ `replace` changes the first match; `replaceAll` changes every match.
- ◆ `split → map → join` transforms text word by word.
- ◆ You'll *read* a few regular expressions in the lab: `/\s+/g` = "one or more spaces, everywhere".

◆ WORKED EXAMPLE

```
function titleCase(str) {
  return str
    .split(" ")
    .map(word => word[0].toUpperCase() + word.slice(1).toLowerCase())
    .join(" ");
}
titleCase("the QUICK brown FOX"); // "The Quick Brown Fox"

"42".padStart(6, "0"); // "000042"
"report-2026.pdf".endsWith(".pdf"); // true
```

◆ YOUR TURN – TEXT FORMATTING UTILITY

Open `l12-strings/starter/`. Write `truncate`, `slugify`, `countWords`, `initials`, and `maskEmail`. Try them in the order `truncate → maskEmail → initials → countWords → slugify`. List the string methods each one needs:

◆ SELF-CHECK

Q1

Are JavaScript strings mutable or immutable?

- A) Mutable — you can change characters in place
- B) Immutable — methods always return new strings
- C) It depends on const vs. let
- D) Mutable with template literals, immutable otherwise

Q4

What does `"Hello, World!".slice(-6)` return?

- A) "Hello,"
- B) "orld!"
- C) An error — indexes can't be negative
- D) "World!"

Q5

What does `"7".padStart(3, "0")` return?

- A) "007"
- B) "700"
- C) "0007"
- D) 7

◆ JOURNAL

Look at some text on your own site — a title, a name, a date — and describe a string function you could write to clean or format it (slug, initials, truncation, padding). Predict what `"JavaScript".slice(-6)` returns before you try it. Which string method do you think you'll use most, and which still confuses you?

◆ NOTES

YOU FINISHED MODULE 3 🎉

You can now store lists in arrays, model real things as objects, transform data with map, filter, and reduce, and clean and format text. In Module 4 you'll connect all of it to the web page itself through the DOM.

The DOM

Your working notebook for the four DOM lessons. Follow along, do the exercises in the space provided, and check yourself with the quizzes. This is the module where your JavaScript starts changing the page.

Lessons 13–16

4 lessons

Name: _____

HOW TO USE THIS WORKBOOK

Each lesson has **Key Ideas** (the reference to keep), a **Worked Example** (read it), **Your Turn** (do it — write your code in the boxes), a **Self-Check** quiz, and **Notes**. The lined boxes are for you to write real code and answers. Self-check questions come from the lesson quizzes and keep the lesson's question numbers, so a set may read Q1 · Q4 · Q5. After each lesson, answer the **Journal** prompt. It takes five minutes and makes the learning stick.

MODULE LAB TRACKER

Each lesson has its own small page to build. Tick each lab as you finish it:

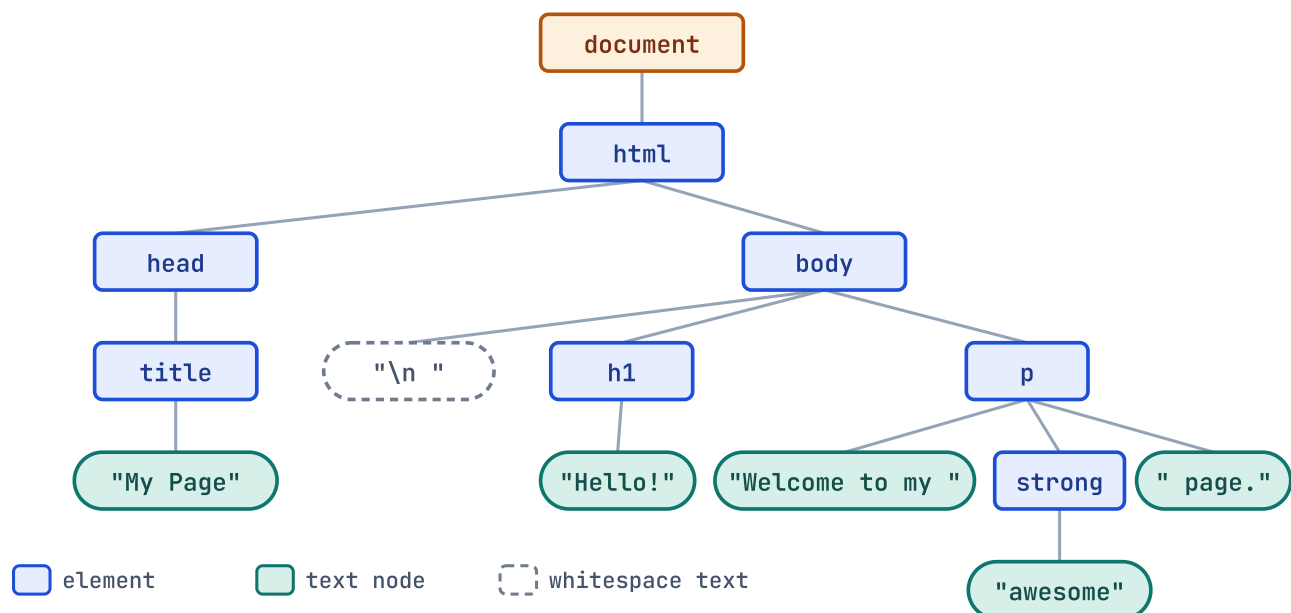
- ◆ ☐ **L13** — DOM Explorer: all 10 console tasks + the 3 "think about it" questions
- ◆ ☐ **L14** — Profile Card Editor: tasks 1–9 (+ the `updateProfile()` bonus)
- ◆ ☐ **L15** — Task List: add, complete, delete with one delegated listener (+ keyboard bonus)
- ◆ ☐ **L16** — Notification System: create, auto-remove, close, clear (+ batch bonus)

◆ BY THE END YOU CAN

- ◆ Explain what the DOM is and how the browser builds it from HTML
- ◆ Name the key objects: `window`, `document`, elements and nodes
- ◆ Move around the tree with parent, child and sibling properties
- ◆ Tell the live DOM (Elements panel) from the HTML source (View Source)

◆ KEY IDEAS – BLUEPRINT, BUILDING, CONTRACTOR

- ◆ **HTML is the blueprint, the DOM is the building, JavaScript is the contractor** who remodels it after it's built.
- ◆ `document` is your entry point to the page. `window` is the browser window that holds everything.
- ◆ Elements are node type 1; text is type 3. The line breaks between your tags are text nodes too.
- ◆ Use the **Element** properties: `parentElement`, `children`, `firstElementChild`, `nextElementSibling`. They skip the whitespace.
- ◆ Console changes live in memory. Refresh and they're gone; View Source never shows them.



The DOM tree for a small page. Line breaks between tags become whitespace text nodes (dashed).

◆ WORKED EXAMPLE

```
const list = document.querySelector("ul");
list.children.length;           // how many <li> elements
list.firstChild;                // the first <li>
list.firstChild;                // #text – the whitespace before it!
list.firstChild.nextSibling;    // the second <li>
list.parentElement;            // whatever contains the list
```

◆ YOUR TURN – DOM EXPLORER

Open the practice page, press F12, and work through the worksheet tasks in the Console. Write the line that walks from `document.documentElement` down to the `<h1>`, and what `#family.firstChild` returned:

◆ SELF-CHECK

Q1

What is the DOM?

- A) The HTML file on the server
- B) An in-memory tree of objects representing the page
- C) A CSS styling system
- D) A JavaScript library

Q4

Your JavaScript adds a new `<p>` to the page. Where will you see it?

- A) In View Source (Ctrl + U)
- B) In your HTML file in VS Code
- C) In the DevTools Elements panel
- D) Nowhere — it only exists in memory

Q5

For a `<ul>` written on several indented lines, why might `list.firstChild` not be the first `<li>`?

A) firstChild only works on the document

B) Because it returns the last child instead

C) Because lists have no children

D) The whitespace before the first `<li>` is a text node, and firstChild includes text nodes

◆ JOURNAL

Open one of your own pages from the HTML & CSS course, then sketch its DOM tree from `document` down to one paragraph. Explain in your own words why a change you make in the console shows up in the Elements panel but not in View Source. What about parents, children, and siblings (or element vs. node properties) are you still unsure about?

◆ NOTES

◆ BY THE END YOU CAN

- ◆ Select elements with `querySelector`, `querySelectorAll` and `getElementById`
- ◆ Change text safely with `textContent`
- ◆ Add, remove and toggle classes with `classList`; set inline styles
- ◆ Read and set attributes and `data-*` values

◆ KEY IDEAS

- ◆ If you know CSS selectors, you know `querySelector`. No match returns `null`.
- ◆ `querySelectorAll` returns a `NodeList`: `forEach` yes, `map` no — spread it with `[...list]`. It's a **snapshot**; re-query after adding elements.
- ◆ `textContent` for anything a user typed. `innerHTML` parses HTML, so user text in it can run an attacker's code (XSS).
- ◆ Prefer classes over inline styles. `element.style` only reads inline styles; use `getComputedStyle` to read what's actually applied.
- ◆ `data-item-id` in HTML is `dataset.itemId` in JavaScript, and its value is always a string.

◆ WORKED EXAMPLE

```
const card = document.querySelector("#card");
card.querySelector("h2").textContent = "Ray de la Paz";
card.classList.add("highlight");
card.classList.toggle("dark");
console.log(card.dataset.userId); // "101"
card.dataset.status = "active"; // adds data-status="active"
document.querySelector("#website").href = "https://example.com";
```

◆ YOUR TURN — DYNAMIC PROFILE CARD EDITOR

Complete tasks 1–9 in `script.js`. After task 6, click **Toggle dark mode**: what happens to the background, and why? Write your fix, then your `updateProfile()` bonus:

◆ SELF - CHECK

Q1

What does `querySelector` return if no element matches?

- A) undefined
- B) An empty NodeList
- C) null
- D) It throws an error

Q2

Why should you avoid `innerHTML` with user input?

- A) It doesn't work in all browsers
- B) It can execute malicious scripts (XSS attacks)
- C) It's too slow for dynamic content
- D) It removes existing CSS styles

Q5

Your stylesheet sets `h1 { font-size: 2rem; }`. What does `heading.style.fontSize` return if no inline style was set?

- A) "2rem"
- B) "32px"
- C) undefined
- D) "" (an empty string)

◆ JOURNAL

Pick one element on a page you built in the HTML & CSS course and write the lines of JavaScript that would select it and change its text, a class, and one attribute. Then explain in your own words why `textContent` is safe for a visitor's name but `innerHTML` isn't. Which of these tools are you still unsure when to use?

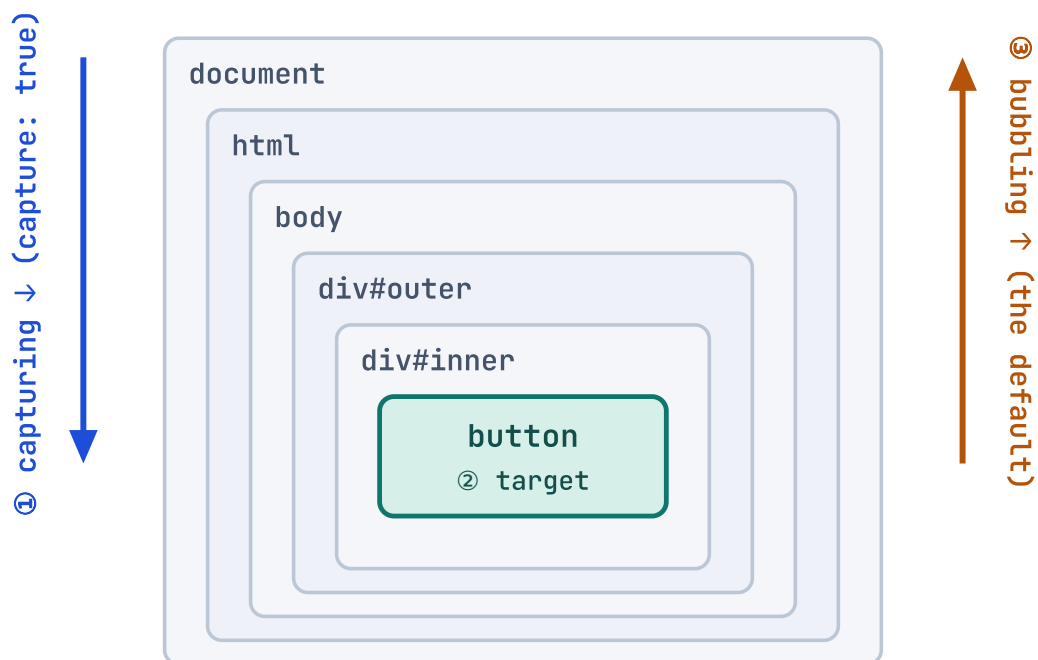
◆ NOTES

◆ BY THE END YOU CAN

- ◆ Respond to clicks, typing, key presses and form submits with `addEventListener`
- ◆ Use the event object (`target` , `key`) and `preventDefault()`
- ◆ Explain bubbling and capturing
- ◆ Handle many children with one delegated listener; remove listeners

◆ KEY IDEAS – THE DOORBELL

- ◆ The **event** is someone pressing the doorbell; the **listener** is what you decide to do when it rings.
- ◆ Pass the function, don't call it: `addEventListener("click", handleClick)` — no `()`.
- ◆ `event.target` = what was actually clicked. `event.currentTarget` = where the listener lives.
- ◆ `event.preventDefault()` stops the browser's own behavior, like a form reloading the page.
- ◆ Events **bubble up** from the target to its ancestors. **Delegation** uses this: one listener on the parent + `event.target.closest(".btn")`.
- ◆ `focus` , `blur` , `mouseenter` and `mouseleave` don't bubble, so you can't delegate them.



A click travels down (capturing), reaches the target, then bubbles back up. Listeners run on the way up unless you pass `{ capture: true }`.

◆ WORKED EXAMPLE

```

form.addEventListener("submit", (event) => {
  event.preventDefault();           // no page reload
  const text = taskInput.value.trim();
  if (!text) return;
  // ...build the <li>, put text in with textContent
});

taskList.addEventListener("click", (event) => { // ONE listener
  const item = event.target.closest(".task-item");
  if (!item) return;
  if (event.target.closest(".delete-btn")) item.remove();
});

```

◆ YOUR TURN – INTERACTIVE TASK LIST

Handle the form submit, build each task, delegate complete/delete from `#task-list`, and write `updateCounter()`. First, **predict**: in the lesson's bubbling example, what order do the three messages print? Then write your delegated click handler:

◆ SELF-CHECK

Q2

What is `event.target` ?

- A) The element the listener is attached to
- B) The element that was actually clicked or interacted with
- C) The type of event that occurred
- D) The parent element of the event

Q4

What is the main advantage of event delegation?

- A) Events fire faster on child elements
- B) It prevents event propagation automatically
- C) One listener handles events for all current and future children
- D) It gives you access to more event properties

Q6

A delegated listener on a list checks `event.target.matches(".delete-btn")`, but clicking the  `<span>` inside the button does nothing. What's the fix?

- A) Use `event.target.closest(".delete-btn")`
- B) Call `event.stopPropagation()` in the span
- C) Add `{ capture: true }` to the listener
- D) Spans can't be clicked, so remove the icon

◆ JOURNAL

Before you click, predict the order of the console messages when you click the button inside #inner in the bubbling example, then explain in your own words how event delegation uses that same bubbling to handle tasks that didn't exist when the page loaded. Name one thing on your own site that could respond to a click, key press, or form submit. What about `event.target`, `preventDefault()`, or delegation are you still unsure about?

◆ NOTES

◆ BY THE END YOU CAN

- ◆ Create an element, configure it, and insert it into the page
- ◆ Insert with `append` / `prepend` (children) and `before` / `after` (siblings)
- ◆ Remove, replace, clear and clone elements
- ◆ Batch many inserts with a `DocumentFragment`

◆ KEY IDEAS – CREATE, CONFIGURE, INSERT

- ◆ `document.createElement("div")` makes an element **in memory**. It isn't on the page until you insert it.
- ◆ An element lives in **one place**. Appending an existing element **moves** it; use `cloneNode(true)` for a copy.
- ◆ `el.remove()` deletes; `el.replaceWith(other)` swaps; `parent.replaceChildren()` empties.
- ◆ User text → `createElement` + `textContent`. Keep `innerHTML` for fixed markup you wrote yourself. `innerHTML +=` rebuilds everything and wipes listeners.
- ◆ A `DocumentFragment` collects elements off-screen; appending it inserts them all in one step and leaves it empty.

◆ WORKED EXAMPLE

```
function showNotification(type, message) {
  const note = document.createElement("div");
  note.classList.add("notification", type);

  const text = document.createElement("span");
  text.textContent = message;           // safe

  const close = document.createElement("button");
  close.textContent = "x";
  close.setAttribute("aria-label", "Dismiss notification");

  note.append(text, close);
  area.prepend(note);                  // now it's visible
  setTimeout(() => note.remove(), 5000);
}
```

◆ YOUR TURN – DYNAMIC NOTIFICATION SYSTEM

Write `showNotification()`, delegate the control buttons and the close buttons, clear everything with `replaceChildren()`, and keep the count up to date. Bonus: the "b" key adds five at once with a `DocumentFragment`. Sketch your delegated close handler and your batch loop:

◆ SELF - CHECK

Q1

What does `document.createElement("div")` return?

- A) A new `<div>` that is immediately visible on the page
- B) A new `<div>` element in memory, not yet in the DOM
- C) A string of HTML
- D) A reference to an existing `<div>` on the page

Q2

What happens if you `append` an element that's already in the DOM?

- A) It creates a copy in the new location
- B) It moves from its old position to the new one
- C) It throws an error
- D) Nothing — the element stays where it was

Q5

Why use a `DocumentFragment` when adding many elements?

- A) It prevents XSS attacks
- B) It keeps a backup of the elements
- C) It batches many insertions into one DOM update
- D) It's the only way to add multiple elements

◆ JOURNAL

Imagine adding a "guestbook" to your own site where visitors type a message that appears on the page. Describe, step by step, how you'd build each new entry with `createElement`, `textContent`, and `append`, and explain in your own words why pasting the visitor's message into `innerHTML` would be dangerous. Looking back over all of Module 4, which DOM skill are you still unsure about?

◆ NOTES

YOU FINISHED MODULE 4 🎉

You can now read the document tree, select and change any element, respond to clicks, keys and form submits, and build or remove elements on the fly — safely. In Module 5 you'll put the whole toolkit to work on real interactive features.

Building Interactive Pages

Your working notebook for the four lessons where you build real features. Follow along, do the exercises in the space provided, and check yourself with the quizzes. By the end of the module you'll have four working mini-apps.

Lessons 17-20

4 lessons

Name: _____

HOW TO USE THIS WORKBOOK

Each lesson has **Key Ideas** (the reference to keep), a **Worked Example** (read it), **Your Turn** (do it and write your code in the boxes), a **Self-Check** quiz, and **Notes**. Use the lined boxes for real code and answers. Self-check questions come from the lesson quizzes and keep the lesson's question numbers, so a set may read Q1 · Q3 · Q4. After each lesson, answer the **Journal** prompt. It takes five minutes and helps the learning stick.

MODULE PROJECT TRACKER

This module has **four mini-apps**, one per lesson. Tick each one when it works:

- ◆ ☐ **L17**: registration form that validates as you go and won't submit bad data
- ◆ ☐ **L18**: Pomodoro timer with start / pause / reset, presets, and an animated ring
- ◆ ☐ **L19**: to-do list that is still there after you refresh the page
- ◆ ☐ **L20**: GitHub user search with loading, error, and profile states

◆ BY THE END YOU CAN

- ◆ Read what users type with `.value` (and `.checked` for checkboxes)
- ◆ Stop the page reload with `event.preventDefault()`
- ◆ Check required, length, format (regex), and matching fields
- ◆ Validate on `blur`, then give live feedback on `input`
- ◆ Explain why the server must check the data again

◆ KEY IDEAS

- ◆ `.value` is **always a string**, even for number inputs. Convert with `Number()`.
- ◆ Listen for `submit` on the **form**, and call `preventDefault()` on the first line.
- ◆ **Touched fields**: validate when the user leaves a field (`blur`), then re-check on every keystroke (`input`), but only after that first blur.
- ◆ Show errors as **text plus color**, with `role="alert"` so screen readers announce them.
- ◆ Client-side validation is for **user experience**. Anyone can bypass it, so the **server always validates again**.

◆ WORKED EXAMPLE

```
form.addEventListener("submit", (event) => {
  event.preventDefault(); // no reload
  const email = form.elements.email.value.trim();
  if (!/^[^\s@]+@[^\s@]+\.[^\s@]+$/.test(email)) {
    showError(form.elements.email, "Please enter a valid email");
    form.elements.email.focus();
    return;
  }
  console.log("Valid. Send it to the server, which checks again.");
});
```

◆ YOUR TURN – REGISTRATION FORM WITH REAL-TIME VALIDATION

Complete TODOs 1–5: the `showError` / `showSuccess` helpers, four validators, a 0–5 password strength meter, blur-then-input wiring, and the submit handler. Then test it: submit it empty, type a weak password, and enter a confirm password that doesn't match. Write your `validateConfirm` function here first:

◆ SELF-CHECK

Q2

What does `event.preventDefault()` do in a submit handler?

- A) Stops the event from bubbling up to parent elements
- B) Clears all the form fields
- C) Stops the browser from reloading the page or navigating away
- D) Removes the event listener after it fires once

Q4

When is the `blur` event useful for form validation?

- A) When you want to validate on every keystroke
- B) When you want to validate after the user finishes with a field and moves on
- C) When the form is submitted
- D) When the user first clicks into a field

Q5

Why is client-side validation not sufficient on its own?

- A) It doesn't work in most browsers
- B) It's too slow for real-time feedback
- C) Users can bypass it through DevTools, disabling JS, or direct API calls
- D) JavaScript can't handle regular expressions for patterns

◆ JOURNAL

Pick a form on a site you use (or the contact form on your own site). List each field and the rule you'd check for it, and decide for each one: HTML attribute, custom JavaScript, or both? In your own words, why must the server check the data again even after your JavaScript approves it? What part of validation are you still unsure about?

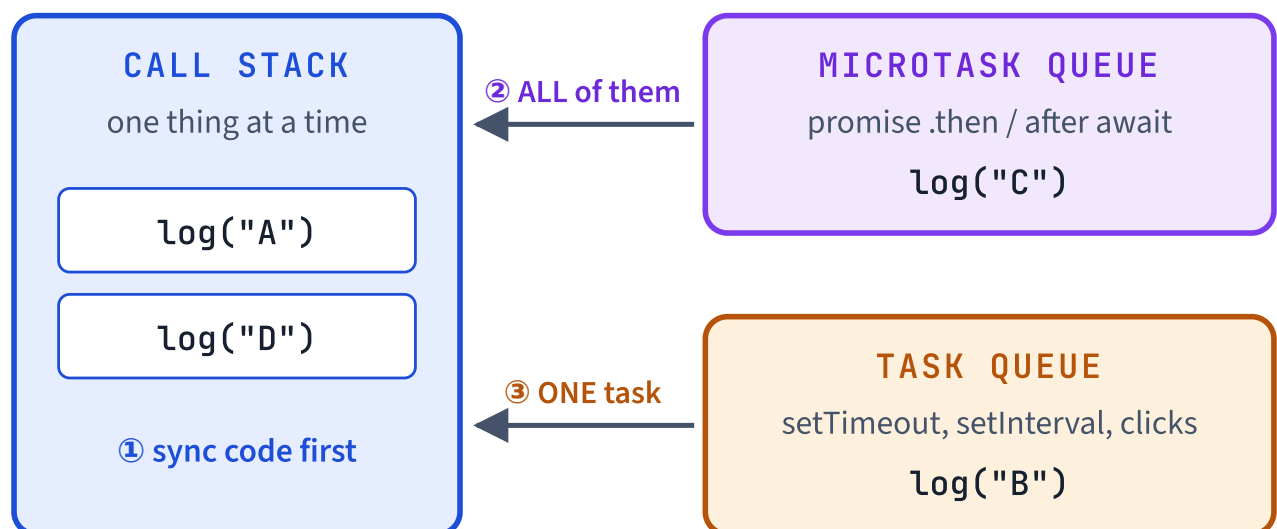
◆ NOTES

◆ BY THE END YOU CAN

- ◆ Run code later with `setTimeout` and cancel it with `clearTimeout`
- ◆ Repeat code with `setInterval` and stop it with `clearInterval`
- ◆ Tell **debounce** (wait for a pause) from **throttle** (at most once per window)
- ◆ Explain why `setTimeout(fn, 0)` doesn't run "right now"
- ◆ Animate with `requestAnimationFrame` or by toggling a CSS class

◆ KEY IDEAS – THE EVENT LOOP

- ◆ Pass the function, don't call it: `setTimeout(show, 1000)`, not `setTimeout(show(), 1000)`.
- ◆ Save the **ID** a timer returns. You need it to cancel the timer.
- ◆ JavaScript does one thing at a time. **Sync code runs first**, then **all promise callbacks** (microtasks), then **one timer or click** (task).
- ◆ A timer's delay is a **minimum**. For accurate countdowns, calculate from `Date.now()` instead of counting ticks.
- ◆ For simple motion, CSS does the animating. JS just adds or removes a class.



Output: A · D · C · B

Sync code first, then every microtask, then one task, and repeat.

◆ WORKED EXAMPLE

```
let intervalId = null; // outside, so pause() can reach it

function start() {
  if (intervalId !== null) return; // no double intervals
  intervalId = setInterval(() => {
    remainingSeconds--;
    updateDisplay(remainingSeconds);
    if (remainingSeconds <= 0) pause();
  }, 1000);
}

function pause() { clearInterval(intervalId); intervalId = null; }
```

◆ YOUR TURN – POMODORO TIMER WITH ANIMATED PROGRESS

Complete TODOs 1–8: DOM references, state, `updateDisplay()` (MM:SS with `padStart`), and the ring's `stroke-dashoffset = CIRCUMFERENCE × (1 - remaining / total)`), start / pause / reset, the presets, and the bonus colors (red under 30 s, green at 0). Write the MM:SS formatting line and the ring formula here:

◆ SELF-CHECK

Q2

What will this code output?

```
console.log("A");
setTimeout(() => console.log("B"), 0);
console.log("C");
```

A) A, B, C

B) A, C, B

C) B, A, C

D) A, B, C (but B might be delayed)

Q3

What's the difference between `setInterval` and recursive `setTimeout` ?

- A) `setInterval` can repeat; `setTimeout` cannot
- B) `setTimeout` can be canceled; `setInterval` cannot
- C) `setInterval` fires every N ms regardless of how long the work takes; recursive `setTimeout` waits for completion first
- D) There's no practical difference

Q5

What is debouncing?

- A) Running a function at most once per time interval
- B) Canceling all pending timers at once
- C) Running a function immediately without delay on each event
- D) Delaying a function until events stop firing, resetting the timer on each new event

◆ JOURNAL

Explain to a friend, in your own words, why `setTimeout(fn, 0)` doesn't run "right now." Then pick one feature for your own site — a search box, a scroll effect, a toast message, or a countdown — and decide which tool fits best: debounce, throttle, `requestAnimationFrame`, or a CSS transition. Why that one? What about the event loop still feels fuzzy?

◆ NOTES

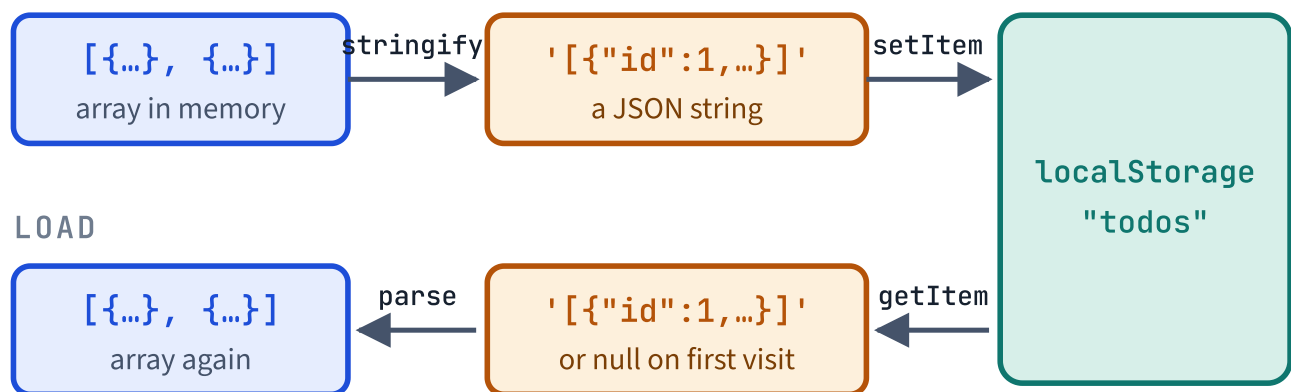
◆ BY THE END YOU CAN

- ◆ Save, read, and delete data with `setItem`, `getItem`, `removeItem`, `clear`
- ◆ Store arrays and objects with `JSON.stringify` and read them back with `JSON.parse`
- ◆ Choose `localStorage` or `sessionStorage`
- ◆ Keep an app's state saved: load → render → change → save → render
- ◆ Name what must **never** go in storage

◆ KEY IDEAS – EVERYTHING IS A STRING

- ◆ Storage holds **strings only**. Saving an object directly stores `"[object Object]"`.
- ◆ `getItem` on a missing key returns `null`, so give it a fallback: `stored ? JSON.parse(stored) : []`.
- ◆ Wrap `JSON.parse` in `try/catch`, because corrupted data throws.
- ◆ `localStorage` lasts until it's deleted and is shared by tabs. `sessionStorage` is per tab and ends when the tab closes.
- ◆ Any script on the page can read storage. **Never** store passwords or tokens there.

SAVE



Strings go in and strings come out. JSON turns them into data.

The JSON round trip: `stringify` to save, `parse` to load.

◆ WORKED EXAMPLE

```
function loadTodos() {
  try {
    const stored = localStorage.getItem("todos");
    return stored ? JSON.parse(stored) : [];
  } catch (error) {
    return []; // corrupted? start fresh
  }
}

function saveTodos(todos) {
  localStorage.setItem("todos", JSON.stringify(todos));
}
```

◆ YOUR TURN – PERSISTENT TO-DO LIST

Complete TODOs 1–7: load/save, `render`, `addTodo`, `toggleTodo` / `deleteTodo`, `updateCount`, one **delegated** click listener on the list, and the first render. Test it by adding two tasks, ticking one, and refreshing. Both should still be there. Sketch your delegated listener (remember that `dataset.id` is a string):

◆ SELF-CHECK

Q1

What does `localStorage.getItem("key")` return if the key doesn't exist?

A) `undefined`

B) `null`

C) An empty string `""`

D) It throws an error

Q3

What is the correct way to store an array in Local Storage?

- A) `localStorage.setItem("data", JSON.stringify(myArray))`
- B) `localStorage.setItem("data", myArray)`
- C) `localStorage.setItem("data", myArray.toString())`
- D) `localStorage.storeArray("data", myArray)`

Q4

How does `sessionStorage` differ from `localStorage` ?

- A) `sessionStorage` has different methods and a smaller size limit
- B) `sessionStorage` sends data to the server; `localStorage` doesn't
- C) `sessionStorage` data is cleared when the tab is closed; `localStorage` persists until deleted
- D) `sessionStorage` is shared across all browser windows; `localStorage` isn't

◆ JOURNAL

Pick one thing on a site you've built (or want to build) that should be remembered between visits — a theme choice, a half-typed message, a list. Would you use `localStorage` or `sessionStorage`, and why? In your own words, explain why you have to `JSON.stringify` it before saving, and name one thing you'd never put in storage. What are you still unsure about?

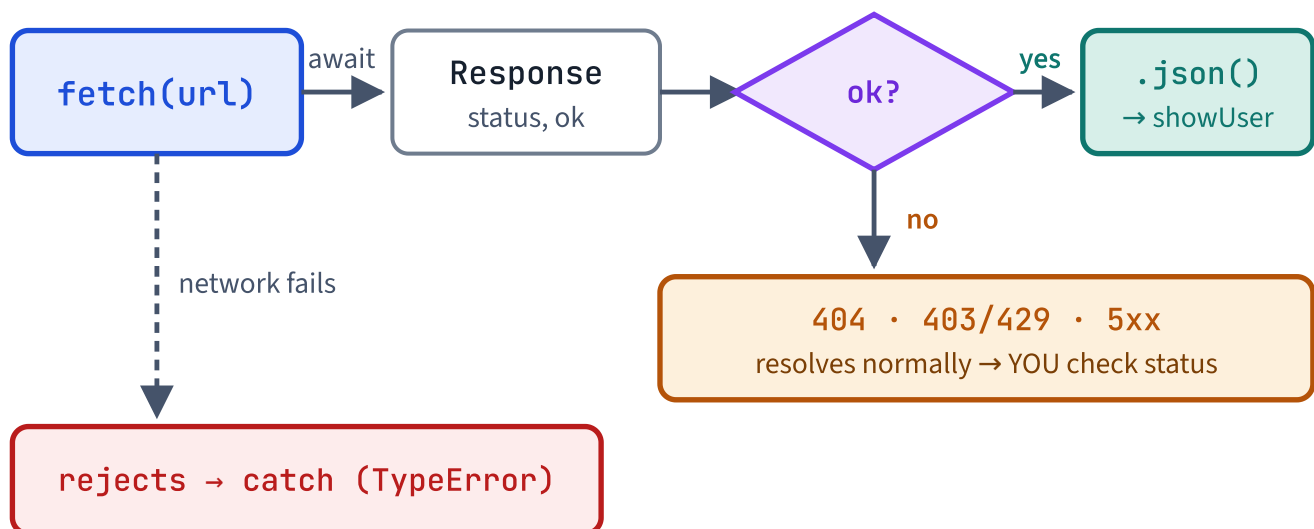
◆ NOTES

◆ BY THE END YOU CAN

- ◆ Explain what an API, an endpoint, and JSON are
- ◆ Describe a Promise's three states: pending, fulfilled, rejected
- ◆ Fetch data with `fetch()` + `await`, and check `response.ok`
- ◆ Handle errors with try/catch and friendly messages
- ◆ Show loading, data, and error states safely

◆ KEY IDEAS – THE FETCH LIFECYCLE

- ◆ `fetch()` gives you a **Promise** right away, and the Response arrives later. `await` waits for it.
- ◆ A 404 or 500 does not land in catch. `fetch` only rejects when the *network* fails. Check `response.ok` / `response.status` yourself.
- ◆ `response.json()` is **also** a Promise, so it needs `await` too.
- ◆ Every request has three UI states: **loading**, **data**, **error**.
- ◆ API text (names, bios) was typed by other people. Put it on the page with `textContent`.



Only a network failure lands in catch. HTTP errors resolve, so check `response.ok`.

◆ WORKED EXAMPLE

```

async function fetchUser(id) {
  try {
    const response = await fetch(
      `https://jsonplaceholder.typicode.com/users/${id}`
    );
    if (!response.ok) {
      throw new Error(`Server error: ${response.status}`);
    }
    return await response.json();
  } catch (error) {
    console.error("Failed to fetch user:", error.message);
    return null;
  }
}

```

◆ YOUR TURN – GITHUB USER SEARCH

Complete TODOs 1–7: `showLoading`, `showError`, `showUser` (all text with `textContent`), and `async searchUser` with separate branches for 404, 403/429 (rate limit), other errors, and network failures. **Heads-up:** without logging in, GitHub allows about 60 requests an hour *per IP address*, and your whole classroom may share one. Work in pairs, don't turn on search-as-you-type, and if you get a 403, fetch `mock-user.json` instead. Sketch your `searchUser` branches here:

◆ SELF-CHECK

Q1

What does `fetch()` return?

- A) The JSON data from the server
- B) A Promise that resolves to a Response object
- C) An XMLHttpRequest object
- D) A string containing the response body

Q2

What happens when `fetch()` gets a 404 response?

- A) The Promise resolves normally with `response.ok` set to `false`
- B) The Promise rejects and `.catch()` runs
- C) It throws an exception automatically
- D) It returns `null`

Q5

What are the three UI states every fetch operation should handle?

- A) Start, finish, and retry
- B) Pending, resolved, and rejected
- C) GET, POST, and DELETE
- D) Loading, data (success), and error

◆ JOURNAL

In your own words, explain why `fetch()` gives you a Promise instead of the data right away, and why a 404 doesn't land in your `catch` block. Then name a free API you'd like to use on your own site and sketch what its loading, data, and error states would look like. What part of async code still feels fuzzy?

◆ NOTES

YOU FINISHED MODULE 5 🎉

You can now validate forms as people type, schedule and animate with timers, save app state so it survives a refresh, and pull live data from an API while handling every possible outcome. In Module 6 you'll combine all of it in a capstone project of your own.

Capstone & Next Steps

Your working notebook for the last two lessons. First you build QuickNote, a complete note-taking app that uses every skill from the course. Then you learn to debug like a professional and plan what to build next.

Lessons 21-22

2 Lessons

Name: _____

HOW TO USE THIS WORKBOOK

L21 is one long guided build: tick each step as your app passes its checkpoint, and use the test checklist at the end. L22 has a bug hunt (fill in the worksheet as you go) and a menu of follow-on projects to choose from. Self-check questions come from the lesson quizzes and keep the lesson's question numbers, so a set may read Q1 · Q2 · Q4. After each lesson, answer the **Journal** prompt. The last one closes the course, so give it the full five minutes.

CAPSTONE TRACKER

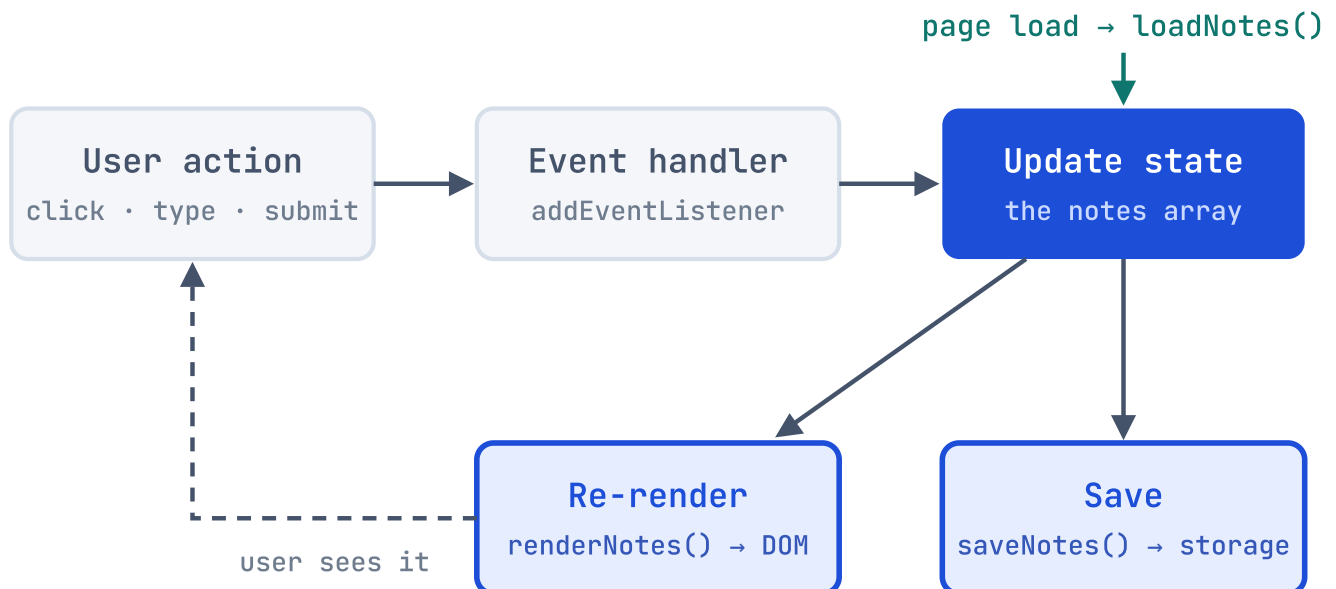
Tick each step when your QuickNote passes its checkpoint:

- ◆ ☐ **Step 1** — starter open in Live Server, no Console errors
- ◆ ☐ **Step 2** — data layer: `createNote()` from the Console lands in Local Storage
- ◆ ☐ **Step 3** — notes render as cards (and the empty state shows when there are none)
- ◆ ☐ **Step 4** — the form creates notes, validates both fields, and flashes "Note saved!"
- ◆ ☐ **Step 5** — debounced search + category filter
- ◆ ☐ **Step 6** — a quote loads, with a fallback when offline
- ◆ ☐ **Step 7** — edit, delete, and N / / Esc shortcuts
- ◆ ☐ **L22** — all seven Snack Budget bugs found and fixed

◆ BY THE END YOU CAN

- ◆ Plan an app first: features, a data model, and how the parts connect
- ◆ Keep a **data layer** (create, read, update, delete + `localStorage`) separate from **rendering**
- ◆ Use **one form** for creating and editing, with validation
- ◆ Add debounced search, a filter, a fetched quote with a fallback, and keyboard shortcuts
- ◆ Put user text on the page safely with `textContent`

◆ KEY IDEAS – ONE LOOP RUNS THE WHOLE APP



QuickNote's loop: user action → update state → save → re-render.

- ◆ **Plan before you code.** A feature list, the shape of one note object, and the loop above save hours of rework.
- ◆ The **data layer** (`loadNotes`, `saveNotes`, `createNote`, `updateNote`, `deleteNote`, `getNotes`) knows nothing about the DOM. `renderNotes()` rebuilds the grid from the data every time.
- ◆ **One form, two modes:** an empty hidden `#note-id` means "create"; an id means "edit."
- ◆ **Event delegation:** the cards are rebuilt on every render, so one click listener on `#notes-grid` handles every  and .

- ◆ **Graceful degradation:** if the quote API fails, show a fallback and keep going. Everything the user typed goes in with `textContent`.

◆ WORKED EXAMPLE – ONE FORM, TWO MODES

```
noteForm.addEventListener("submit", (event) => {
  event.preventDefault();
  if (!validateForm()) return;
  const editId = noteIdInput.value;           // "" or "17133000000000"
  if (editId) {
    updateNote(Number(editId), titleInput.value, bodyInput.value, categorySelect.value);
    showSaveStatus("Note updated!");
  } else {
    createNote(titleInput.value, bodyInput.value, categorySelect.value);
    showSaveStatus("Note saved!");
  }
  hideForm();
  renderNotes(searchInput.value, filterCategory.value);
});
```

Read it as the loop: the submit is the user action; `createNote` / `updateNote` update state and save; `renderNotes` re-renders. `Number()` is needed because input values are always strings.

◆ YOUR TURN – BUILD QUICKNOTE

Open `starter/quicknote.html` with Live Server and follow along, TODO by TODO. Before Step 2, sketch **one note object** (its properties and example values):

When you finish, test your app. Tick each one that works:

- ◆ ☐ Add a note in each category (newest first)
- ◆ ☐ Save with both fields empty → *both* error messages
- ◆ ☐ Edit a note (button says "Update Note")
- ◆ ☐ Delete a note (confirm, fade out, count updates)
- ◆ ☐ Delete the note you're editing → the form closes
- ◆ ☐ Search waits for you to pause; the category filter is instant
- ◆ ☐ Reload → notes are still there
- ◆ ☐ Quote loads; offline shows the fallback and nothing breaks

- ◆ ☐ N new note (but typing "n" in a field doesn't), / search, Esc close
- ◆ ☐ A title like `<img src=x onerror=alert(1)>` shows as plain text

Bugs you hit and how you fixed them:

Stretch: circle one to try next — dark mode · export/import · drag & drop · Markdown preview · multi-tab sync · word count · confirmation modal · pin notes.

◆ SELF-CHECK

Q1

QuickNote uses one form for both creating and editing notes. How does the submit handler know which one to do?

- A) It reads the text on the Save button
- B) It checks whether the hidden `note-id` input has a value
- C) It checks whether the notes array is empty

Q2

Why does `renderNotes()` put the note's title and body on the page with `textContent` instead of `innerHTML` ?

- A) `textContent` is much faster, so the grid renders quicker
- B) `innerHTML` can't display text loaded from localStorage
- C) So anything the user typed is shown as plain text and never run as HTML or script

Q4

Why does QuickNote attach ONE click listener to `#notes-grid` instead of a listener on every edit and delete button?

- A) Browsers only allow one click listener per page
- B) `closest()` only works on container elements
- C) So clicks on the buttons are ignored
- D) The cards are rebuilt on every render, so one listener on the parent (event delegation) handles every card, old or new

◆ JOURNAL

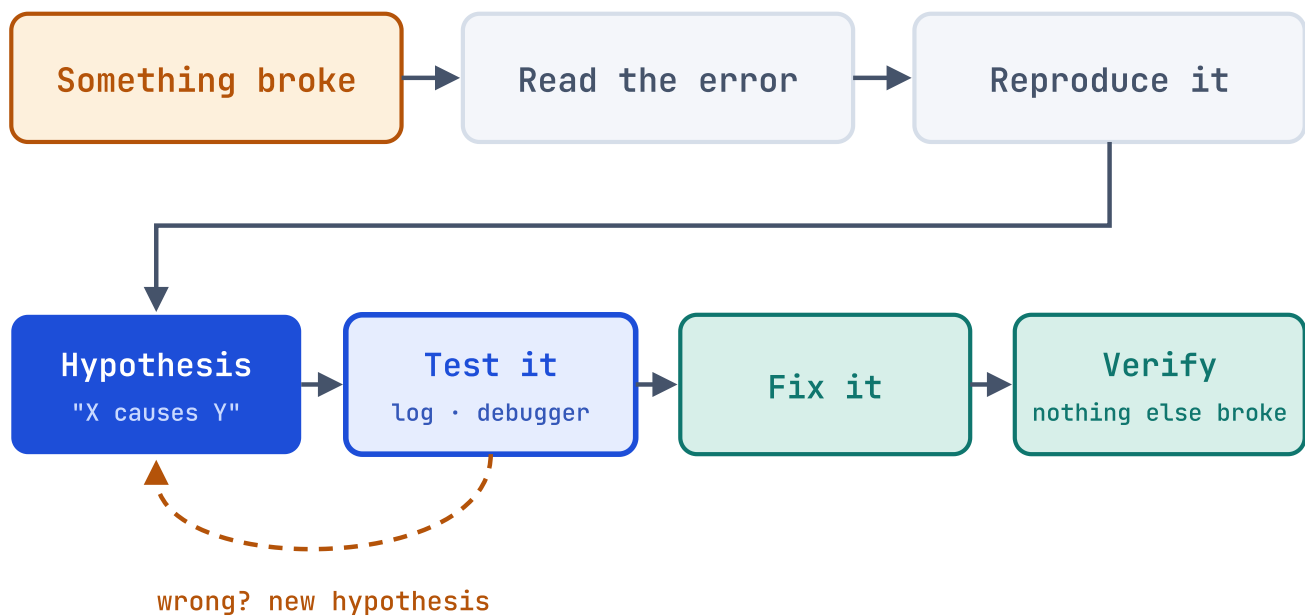
QuickNote runs on one loop: user action → update state → save → re-render. Pick one feature (deleting a note, say) and trace it through that loop in your own words, naming the functions it calls along the way. What's one feature you'd add to make QuickNote your own, and which part of the app's flow are you still unsure about?

◆ NOTES

◆ BY THE END YOU CAN

- ◆ Read an error message for **what** went wrong and **where**
- ◆ Debug with `console.table`, `debugger` and breakpoints, one hypothesis at a time
- ◆ Organize code by concern with clear names and small functions
- ◆ Spot the common pitfalls and read modern syntax like `?.` and `??`
- ◆ Choose what to learn and build next

◆ KEY IDEAS – DEBUG WITH A LOOP, NOT WITH LUCK



The debugging loop – change one thing at a time, on purpose.

ERROR SAYS...	USUALLY MEANS	TRY
<code>Cannot read properties of null</code>	<code>querySelector</code> returned <code>null</code>	Check the selector; does the element exist yet?
<code>X is not a function</code>	Calling something that isn't a function	Check spelling, import, variable type
<code>X is not defined</code>	The variable doesn't exist in this scope	Check spelling, scope, declaration order

ERROR SAYS...	USUALLY MEANS	TRY
Unexpected token	Syntax error (missing bracket, comma)	Check the line before the error line
Cannot read properties of undefined	A nested property's parent is missing	Log the data's shape; use <code>?.</code>

- `debugger;` only pauses when DevTools is **open**. Or click a line number in **Sources** to set a breakpoint.
- **Pitfalls:** queries can return `null` · `fetch` isn't finished on the next line · use `===` · spread copies are **shallow** (`structuredClone` is deep) · add delegated listeners **once** · test the edge cases (empty, zero, offline, first visit).
- `??` falls back only for `null` / `undefined`; `||` also replaces `0`, `""` and `false`.
- **Organize by concern** (data · rendering · events), name things for what they do, keep functions small. Split big projects into modules with `<script type="module">`; they need a server like Live Server.

WORKED EXAMPLE – READING AN ERROR

```
// Uncaught TypeError: Cannot read properties of null (reading 'textContent')
//    at renderUser (app.js:45:23)

// WHAT:  reading .textContent on something that's null
// WHERE: app.js line 45, inside renderUser
// WHY:   querySelector probably found nothing
const el = document.querySelector("#user-name");
if (el) el.textContent = name;           // or: check the selector!
```

YOUR TURN – BUG HUNT: SNACK BUDGET

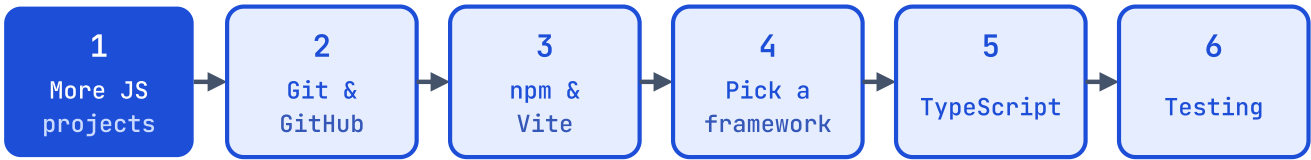
Open `122-debugging/starter/` with Live Server, open the Console, and find all **seven** bugs. Fix one, save, reload, repeat. Log each one:

#	WHAT I SAW (ERROR OR BEHAVIOR)	LINE	MY HYPOTHESIS	MY FIX
1				
2				
3				
4				
5				

#	WHAT I SAW (ERROR OR BEHAVIOR)	LINE	MY HYPOTHESIS	MY FIX
6				
7				

Which bug took longest, and what would have found it faster?

◆ YOUR LEARNING PATH



A practical order – each step builds on the one before.

- ◆ **80/20:** 20% reading and watching, 80% building. Don't tutorial-hop; finish one resource and build something with it.
- ◆ Starting a framework project today? `npm create vite@latest` (or Next.js / Nuxt / SvelteKit). Skip `create-react-app`; it was deprecated in 2025.
- ◆ **Portfolio:** three polished, **deployed** projects with READMEs beat ten half-finished ones.

◆ FOLLOW-ON PROJECT MENU – PICK YOUR NEXT BUILD

These are the lesson's project ideas, from ★ (a weekend) to ★★★ (a real stretch). Tick one, or write your own in the last row.

PICK	LEVEL	PROJECT	SKILLS PRACTICED
☐	★	Pomodoro timer with sound	Timers, Audio API, localStorage
☐	★	Color palette generator	Random values, DOM, clipboard API
☐	★★	Weather app with a real API	Fetch, API keys, geolocation
☐	★★	Quiz app with scoring	Arrays, state management, timers
☐	★★	Expense tracker with charts	Forms, localStorage, canvas/SVG
☐	★★★	Kanban board with drag & drop	Drag & Drop API, complex state

PICK	LEVEL	PROJECT	SKILLS PRACTICED
☐	★ ★ ★	Chat app with WebSockets	Real-time data, Node.js back-end
☐	★ ★ ★	Markdown editor with live preview	Text processing, content rendering
☐	?	My own idea:	

Plan it like QuickNote. Features (3–5), the shape of one data object, which lessons' skills it uses, and your very first step:

◆ SELF-CHECK

Q1

The console shows `Cannot read properties of null (reading 'textContent')`. What's the most likely cause?

A) `textContent` is misspelled

B) The browser doesn't support `textContent`

C) `querySelector` didn't find the element, so it returned `null`

Q2

A player's score is `0`. What does `const shown = player.score ?? 10; set shown` to?

A) `0`

B) `10`

C) `false`

Q3

You copy an object with `const copy = { ...original };` and then push a new item onto `copy.scores` (an array). What happens to `original.scores` ?

- A) Nothing — spread makes a completely separate copy
- B) It changes too, because the nested array is shared
- C) JavaScript throws an error

◆ JOURNAL

Look back at the whole course. Which JavaScript skill felt impossible at the start but makes sense to you now? Pick one project from the "Project Ideas" table (or your own idea) and write down which lessons' skills it would use and the very first step you'd take to build it. What's one topic you're still unsure about and want to revisit before moving on?

◆ NOTES

YOU DID IT — YOU'RE A JAVASCRIPT DEVELOPER

You went from "what is JavaScript?" to a complete app that saves data, talks to an API, and responds to the keyboard. That's the whole journey: **foundations → control flow & functions → data → the DOM → interactive pages → a complete app**. Deploy QuickNote, pick your next project from the menu, and keep building. The difference between where you are today and where you want to be is more time and more projects.